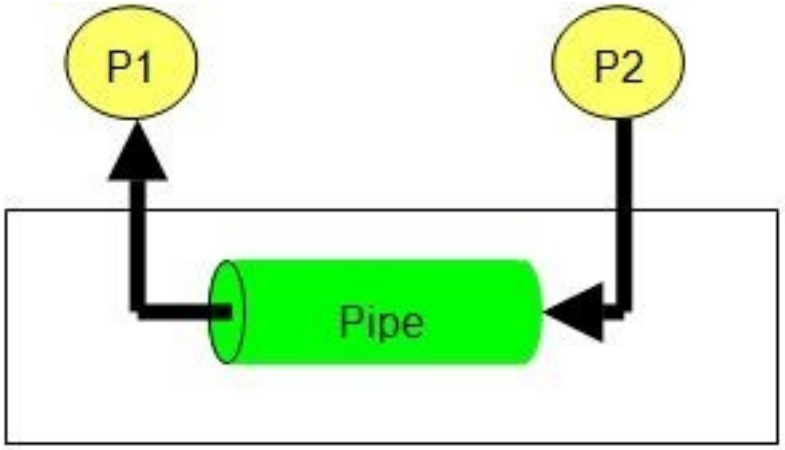
Pràctica 4 i 5. Comunicació entre processos

Comunicació entre processos.

Existeixen diverses formes de comunicació entre processos de Linux: pipes, cues de missatges, semàfors i segments de memòria compartida.

Ens centrem en els **pipes** (canonades en valencià, tuberías en castellà).
Què és un Pipe? És una espècie de fals fitxer que serveix per a connectar dos processos.



Quan P2 vol enviar dades a P1, les escriu en el pipe com si fora un fitxer d'eixida. El procés P1 pot llegir les dades llegint del pipe com si es tractarà d'un fitxer d'entrada (comunicació similar a lectura i escriptura en fitxers).

Com creem un pipe?

#include <unistd.h>
int pipe (int fd[2])

Observa que la funció només rep fd[2], un array de dos enters.
fd[0] Conté el descriptor de lectura.
fd[1] Conté el descriptor d'escriptura.

Si la funció té èxit retorna 0 i si ocorre algun error retorna -1.

Com escriure i llegir del pipe?

*int read (int fd, void *buf, int count);*
*int write(int fd, void *buf, int count);*

- **read()** intenta llegir count bytes del descriptor de fitxer definit en fd, per a guardar-los en el buffer buf. Retornarà el nombre de bytes llegits.
- **write()** En buf li passem el missatge que vulguem escriure, definim la seua grandària en count i especifiquem el fitxer en el qual escriurem en fd.

Veurem un exemple que ens quedarà més clar de l'ús **de fitxers en C** (recorda en C usem open() i close() per a obrir i tancar arxius) (T1S3Exemple7Fitxer.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

void main(void)
{
	char salutacio[] = "Salutacions!!!";
	char buffer[10];
	int fd, bytesllegits;

	// PRIMERA PART: ESCRIC MISSATGE EN EL FITXER
	fd = open("text.txt",O_WRONLY); // OBRE EL FITXER PER ESCRIPTURA
	if (fd==1) {
		printf ("Error a l'obrir l'arxiu\n");
		exit(1);
	}
	printf("Escric la salutació que vaig a guardar en fitxer:%s...\n",salutacio);
	write(fd,salutacio,strlen(salutacio));
	close(fd); // TANQUE EL FITXER

	// SEGONA PART: LLIG MISSATGE DEL FITXER
	fd = open("text.txt",O_RDONLY|O_CREAT); // OBRE EL FITXER PER LECTURA
	printf("Contingut del fitxer\n");
	// LLIG DE BYTE EN BYTE I ELS GUARDE EN buffer
	bytesllegits = read(fd,buffer,1);
	while (bytesllegits!=0) {
		printf("%s",buffer); // ESCRIC BYTE LLEGIT
		bytesllegits = read(fd,buffer,1); // LLIG UN ALTRE BYTE
	}
	close(fd); // TANQUE EL FITXER
	printf("Escric la salutació que he extret del fitxer:%s...\n",salutacio);
}
```

Una vegada hem recordat com escriure fitxer vegem exemples amb pipes.

Exemple

Creem procés amb fork(). El procés fill envia al procés pare mitjançant l'ús de pipes el missatge “Hola papi” en el descriptor d’escriptura fd[1]. El procés pare, mitjançant el descriptor fd[0] llig les dades enviades pel fill. Fixa’t que, a diferència de l’ús en fitxers, la funció de lectura READ és bloquejant. Per tant, llegim tots els bytes d’una, en este cas, 10 (T1S3Exemple8ForkPipeMAL.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
   int fd[2];
   char buffer[10];
   pid_t pid;

   pipe(fd); // CREE EL PIPE
   pid=fork(); // CREE EL PROCÉS FILL
   switch(pid)
   {
      case 1: //ERROR
                        printf("No s'ha pogut crear el procés fill\n");
                        exit(1);

      case 0: // PROCÉS FILL ENVIA
                        printf("El fill escriu al PIPE...\n");
                        write(fd[1],"Hola papi\0",10);
                        break;

      default: // PROCÉS PARE REP
                        wait(NULL); // PARE ESPERA A QUE ACABE EL FILL
                        printf("El pare llig del PIPE...\n");
                        read(fd[0],buffer,10);
                        printf("Missatge llegit: %s\n",buffer);
                        break;
   } // FI SWITCH
}
```

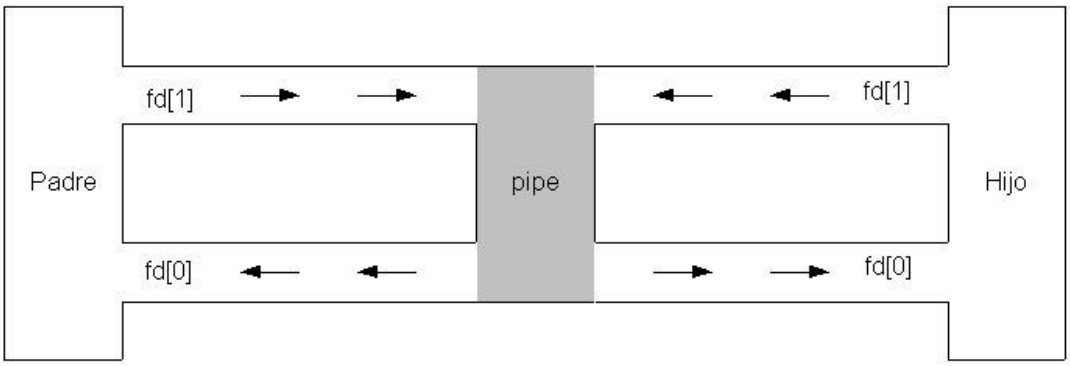
La compilació i execució:

```
pau ~/ $ ./UD1Exemple8ForkPipeMAL
El fill escriu al PIPE...
El pare llig del PIPE...
Missatge llegit: Hola papi
```

El procés que segueix l'exemple és:

1 Crea canonada amb pipe()
2 Crea procés fill.

Quan creem un procés amb fork(), el fill rep còpia de tots els descriptors de fitxers del procés pare (incloent fd[0] i fd[1]). Això ens permet que el procés fill escriga en l'extrem d'escriptura fd[1] i el pare els reba en l'extrem de lectura fd[0].



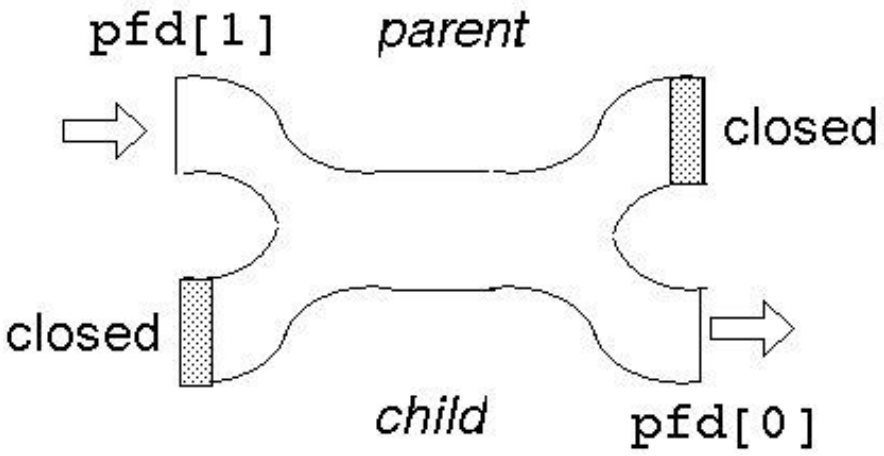
Els processos pare i fill estan units pel pipe (veure figura anterior), però la comunicació és en una única direcció, per tant s'ha de decidir en quina direcció s'envia la informació, del pare al fill o del fill al pare. En compartir-se els descriptors hem d'estar segurs de tancar l'extrem que no ens interessa.

Flux informació de pare a fill.

- El pare ha de tancar el descriptor de lectura fd[0] (no s'usarà)
- El fill ha de tancar el descriptor d'escriptura fd[1] (no s'usarà)

Flux informació de fill a pare.

- El pare ha de tancar el descriptor d'escriptura fd[1] (no s'usarà)
- El fill ha de tancar el descriptor de lectura fd[0] (no s'usarà)



Implementarem la comunicació de la imatge anterior on creem un pipe en el qual el pare envia un missatge al fill, és a dir, el flux de la informació va del pare al fill. El pare ha de tancar el descriptor fd[0] i el fill el fd[1]. El pare escriu en fd[1] i el fill llig de fd[0].

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>

int main()
{
	int fd[2];
	pid_t pid;
	char saludoPadre[]="Bon dia fill\0";
	char buffer[sizeof(saludoPadre)+1];

	pipe(fd); // CREE EL PIPE
	pid=fork(); // CREE EL PROCÉS FILL

	switch(pid) {
		case 1: //ERROR
			printf("No s'ha pogut crear el procés fill\n");
			exit(1);

		case 0: // PROCÉS FILL REP
			close(fd[1]); // TANCA EL DESCRIPTOR D'EIXIDA
			read(fd[0],buffer,sizeof(saludoPadre)+1); // LLIG
			printf("Missatge llegit: %s\n",buffer);
			break;
		default: // PROCÉS PARE ENVIA
			close(fd[0]); // TANCA EL DESCRIPTOR D'ENTRADA
			printf("El pare escriu al PIPE...\n");
			write(fd[1],saludoPadre,strlen(saludoPadre)+1); // ESCRIU
			wait(NULL); //espero al proceso hijo
			break;
	}
	return 0;
}
```

L'execució mostra els següents resultats:

```
pau ~/ $ ./T1S3Exemple9ForkPipeOK
El pare escriu al PIPE...
Missatge llegit: Bon dia fill
```

Pràctica 4.

Realitza un programa en C que cree un pipe en el qual el fill envie un missatge al pare, és a dir, la informació, fluïsca del fill al pare.

L'execució ha de mostrar la següent eixida:

```
pau ~/ $ gcc T1S3P4ForkPipe.c -o T1S3P4ForkPipe
pau ~/ $ ./T1S3P4ForkPipe
 El fill envia al pare...
El pare rep del fill: Hola papi
```

Comunicació en tots dos sentits entre procés avi, pare i fill

Farem que pare i fills puguen enviar i rebre informació, com la comunicació és un únic sentit necessitarem dos pipes fd1 i fd2. Cada procés usarà un pipe per a enviar la informació i un altre pipe per a rebre-la.

Realitzem un esquema gràfic per a tindre clar el que volem implementar

AVI	fd1 --->	**PARE**	fd2 ---->	**NET**
	fd2 ←		fd1 ←	

- L'AVI
  - Envia informació a través de fd1.
  - Rep a través de fd2.
- El PARE
  - Envia informació a través de fd2.
  - Rep la informació a través de fd1.
- El NET
  - Usarà fd1 per a enviar informació
  - Rebrà la informació a través de fd2.

Intenta desenvolupar el programa pel teu compte, si necessites ajuda el codi del programa és (T1S3Exemple10AviPareNet.c):

```
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
#include <string.h>

int main(int argc, char *argv[])
{
   pid_t pid1, pid2;
   int status1, status2;
   int fd1[2], fd2[2];
   pipe(fd1);
   pipe(fd2);

   char salutacioAviPare[]="Salutacions del teu pare.\0";
   char salutacioPareNet[]="Salutacions del teu pare.\0";
   char salutacioNetPare[]="Salutacions del teu fill.\0";
   char salutaciolPareAvi[]="Salutacions del teu fill.\0";
   char buffersalutacioAviPare[strlen(salutacioAviPare)+1];
   char buffersalutacioPareNet[strlen(salutacioPareNet)+1];
   char buffersalutacioNetPare[strlen(salutacioNetPare)+1];
   char buffersalutacioPareAvi[strlen(salutaciolPareAvi)+1];

   pid1=fork();
   if ( pid1== 0 ) { /* PARE */
      pid2=fork();
      if ( pid2== 0 ) { /* NET */
         close (fd1[0]); // TANCA EL DESCRIPTOR D'ENTRADA fd1
         close (fd2[1]); // TANCA EL DESCRIPTOR D'EIXIDA fd2

         read (fd2[0],buffersalutacioPareNet, sizeof(buffersalutacioPareNet)); // LLIG EN EL
PIPE
         printf("NET rep msg del PARE. %s\n", buffersalutacioPareNet);
         printf("NET envia msg al PARE.\n");
```

```
            write (fd1[1],salutacioNetPare, strlen(salutacioNetPare)+1); // ESCRIC EN EL PIPE


            printf("Soc el net (%d, fill de %d)\n", getpid(), getppid());
        }
        else { /*  PARE */
            close (fd2[0]); // TANCA EL DESCRIPTOR D'ENTRADA fd2
            close (fd1[1]); // TANCA EL DESCRIPTOR D'EIXIDA fd1

            read (fd1[0],buffersalutacioAviPare, sizeof(buffersalutacioAviPare)); // LLIG EN EL
PIPE
            printf("PARE rep msg de l'AVI. %s\n", buffersalutacioAviPare);
            printf("PARE envia msg al FILL.\n");
            write (fd2[1],salutacioPareNet, strlen(salutacioPareNet)+1); // ESCRIC EN EL PIPE
            read (fd1[0],buffersalutacioNetPare, sizeof(buffersalutacioNetPare)); // LLIG EN EL
PIPE
            printf("PARE rep msg del NET. %s\n", buffersalutacioNetPare);
            printf("PARE envia msg a l'AVI.\n");
            write (fd2[1],salutaciolPareAvi, strlen(salutaciolPareAvi)+1); // ESCRIC EN EL PIPE

            waitpid(pid2, &status2, 0);
            printf("Soc el pare (%d, fill de %d)\n", getpid(), getppid());
        }
    }
    else { /* AVI */
         /* ESPEREM AL PARE (PARE DEL NET) ACABE, status 0 */
        close (fd1[0]); // TANCA EL DESCRIPTOR D'ENTRADA fd1
        close (fd2[1]); // TANCA EL DESCRIPTOR D'EIXIDA fd2
        printf("AVI envia msg al PARE\n");
        write (fd1[1],salutacioAviPare, strlen(salutacioAviPare)+1); // ESCRIC EN EL PIPE
        waitpid(pid1, &status1, 0);
        read (fd2[0],buffersalutacioPareAvi, sizeof(buffersalutacioPareAvi)); // LLIG EN EL PIPE
        printf("AVI rep msg del PARE. %s\n", buffersalutacioPareAvi);
        printf("Sóc l'avi (%d, fill de %d)\n",getpid(), getppid());
    }
    return 0;
}
```

La compilació i l'execució:

gcc T1S3Exemple10AviPareNet.c -o T1S3Exemple10AviPareNet
./T1S3Exemple10AviPareNet

AVI envia msg al PARE
PARE rep msg de l'AVI. Salutacions del teu pare.
PARE envia msg al FILL.
NET rep msg del PARE. Salutacions del teu pare.
NET envia msg al PARE.
Soc el net (6347, fill de 6346)
PARE rep msg del NET. Salutacions del teu fill.
PARE envia msg a l'AVI.
Soc el pare (6346, fill de 6345)
AVI rep msg del PARE. Salutacions del teu fill.
Sóc l'avi (6345, fill de 2939)

PIPES amb nom o FIFOS

Els PIPES vistos fins ara estableixen comunicació entre processos (pare-fill), però només podrem comunicar-nos si són família. Els FIFOS permeten comunicar processos que no tenen perquè estar emparentats.

Un FIFO és com un fitxer amb nom. Existeix en el sistema de fitxers, es poden obrir, llegir i escriure per múltiples processos. Les dades escrites en els FIFO es lligen com en una cua, i una vegada llegits no poden ser llegits de nou.

Diferències amb els fitxers:

- ❏ Una operació d'escriptura en un FIFO queda en espera fins que el procés pertinent òbriga el FIFO per a iniciar la lectura.
- ❏ Només es permet l'escriptura d'informació quan un procés vaja a recollir aquesta informació.

Formes d'usar FIFO:

- Executant mknod des del terminal
- Des d'un programa en C usant mknod()

Ús mknod des de línia de comandos.

mknod [opcions**]** nom **p**

L'argument que segueix a nom especifica el tipus de fitxer a construir:

- p per a un FIFO
- b per a un fitxer especial de blocs (amb búffer)
- c per a un fitxer especial de caràcters (sense búfferes)

Quant a les opcions: -m manera, --mode=mode estableix els permisos dels fitxers creats a modo, que és simbòlic com en chmod.

Veurem millor el seu funcionament mitjançant un exemple.

Exemple

Cree fitxer FIFO1

>mknod FIFO1 p

A continuació, execute l'ordre de lectura del FIFO1
>cat FIFO1

No mostra res perquè no hi ha res en la canonada (pipe) i queda en espera. Però i si envie alguna cosa..
Òbric un altre terminal i execute:
>ls > FIFO1

Veurem que el cat s'executa i mostra la informació fruit de l'execució del comando ls.

Veurem el seu funcionament en C.
Per a crear un FIFO en C, es pot fer ús de la crida del sistema mknod() amb els seus respectius arguments:

→ mknod("/tmp/MIFIFO", S_IFIFO|0644, 0);

En aquest cas l'arxiu "/tmp/MIFIFO" es crea com a arxiu FIFO. El segon argument és la manera de creació, que serveix per a indicar-li a mknod() que crea un FIFO (amb S_IFIFO) i posa els permisos d'accés a aqueix arxiu (644 octal, rw-r--r--) a l'igual com es pot fer amb el comando chmod. Finalment, el tercer argument de mknod() s'ignora (en crear un FIFO) llevat que s'estiga creant un arxiu de dispositiu. En aqueix cas, s'hauria d'especificar els números major i menor de l'arxiu de dispositiu.
Les operacions E/S sobre un FIFO són essencialment les mateixes que per a les canonades (pipes) normals, amb una gran excepció. S'ha d'usar una crida del sistema open() o una funció de llibreria per a obrir físicament un canal per a la canonada.

Exemples

Programarem un programa T1S3Exemple11FIFOCrea.c que crea un FIFO i llig la informació del FIFO. (mentre no hi ha informació roman a l'espera) i un altre T1S3Exemple11FIFOEscribe.c que escriu en el FIFO.

T1S3Exemple11FIFOCrea.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <fcntl.h>
#define MIDA_BUFFFER 80
#define TRUE 1

int main(void)
{
        int fp;
        char buffer[MIDA_BUFFFER];
        int nbytes;

        mknod("FIFO1",S_IFIFO|0660,0);

        while(TRUE)
        {
                fp=open("FIFO1",O_RDONLY);
                nbytes=read(fp,buffer,MIDA_BUFFFER-1);
                buffer[nbytes]='\0';
                printf("Cadena rebuda: %s \n",buffer);
                close(fp);
        }

        return 0;
}
```

T1S3Exemple11FIFOEscribe.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>

int main()
{
        int fp;
        char salutacio[]="Una salutació!!!";
        fp=open("FIFO1",1);

        if (fp==-1)
        {
                printf ("ERROR AL ABRIR ARCHIVO\n");
                exit(1);
        }

        printf ("Manant informació al FIFO...\n");
        write (fp,salutacio,strlen(salutacio));
        close(fp);
        return 0;
}
```

Per a provar l'exemple, haurem d'obrir dos terminals per a que siga més fàcil.
En la primera terminal, executem T1S3Exemple11FIFOCrea, el qual crea la FIFO i espera a rebre el que l'altre procés envia. Cap missatge apareix.
En la segona terminal, executem T1S3Exemple11FIFOEscribe, que envia la cadena «Una salutació!!!» i acaba.
Quan hagem executat en la segona terminal, en la primera voràs com apareix el missatge: «Una salutació!!!»
Podem executar a la segona terminal el programa T1S3Exemple11FIFOEscribe tantes vegades com vullguem i observaràs com en la primera terminal apareixen idèntics nombre de missatges.

Pràctica 5

Realitzar el problema del productor/consumidor mitjançant FIFOS amb noms T1S3P5Consumidor.c i T1S3P5Productor.c.
El consumidor és el que s'ha d'encarregar de crear i esborrar la canonada.
El productor ha d'escriure en la canonada amb els caràcters proporcionats per l'usuari des de teclat.
Quan l'usuari proporcione un caràcter punt des de teclat, el programa ha d'acabar la comunicació.
El consumidor per la seua part, ha de llegir des de la canonada amb nom fins que no quede res en la canonada.